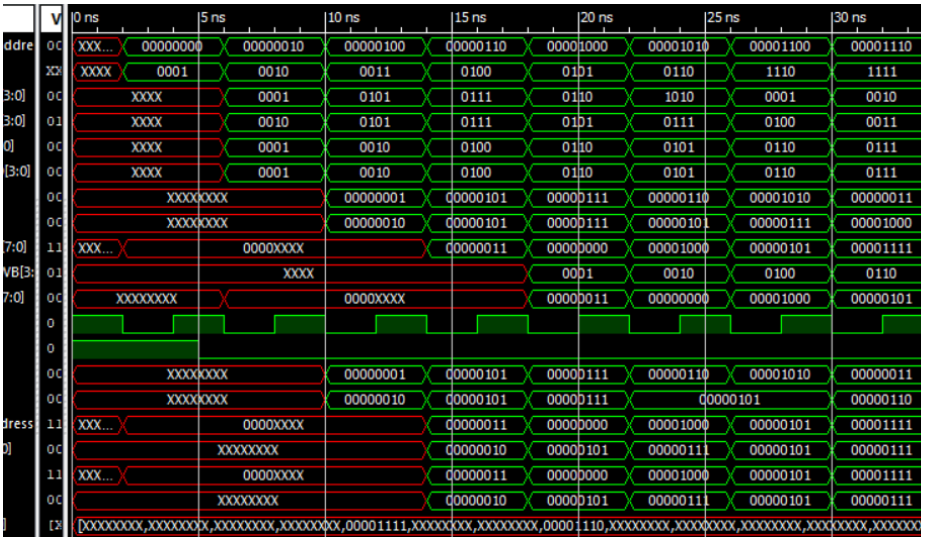
UNIVERSIDAD DE GUADALAJARA
CUCEI

Carrera: Ingeniería en Computación

Materia: Arquitectura de Computadoras

Reporte

PF00-ACT. FINAL TIPO R-DTPR



Integrantes:

Hernández Zúñiga Roberto López López Karla Valeria

Ortiz Segura Emmanuel Alejandro Segura Gutierrez Aaron Ricardo

Maestro: López Arce Delgado Jorge Ernesto

Sección: D03

Fecha: 24 abril 2026

Índice

1. **Introducción**

   1.1 Resumen Video

2. **Objetivos**

   2.1 Objetivo General

   2.2 Objetivos Específicos

3. **Marco Teórico**

   3.1 Instrucciones Tipo R

   3.2 Operador Ternario

   3.3 Proceso de Compilación

4. **Desarrollo**

   4.1 Capturas Simulación

5. **Conclusión**

6. **Bibliografía**

1. Introducción

A lo largo de este reporte, se detalla el proceso de investigación, diseño e implementación del Single Datapath para un procesador que utilizará arquitectura MIPS (Microprocessor without Interlocked Pipelined Stages), ejecutando instrucciones de tipo R.

Un Single Datapath (ruta de datos de ciclo único) es un diseño de procesador, en el que cada instrucción se realiza con un solo ciclo de reloj. Utiliza banco de registros, una ALU (Unidad Aritmético Lógica), y multiplexores para gestionar el flujo de datos e instrucciones, desde su búsqueda, decodificación, ejecución y escritura. Este modelo es muy usado para la enseñanza de la arquitectura RISC, siendo MIPS una microarquitectura simple de este tipo, ideal para comprender la ejecución básica de instrucciones.

1.1 Resumen Video

En el diseño de procesadores, existen 3 tipos para la categorización de instrucciones que facilitan la decodificación dentro de la unidad de control, estos utilizan un formato de 32 bits, a continuación una breve descripción de cada uno:

Tipo R (registro) (utilizado para esta práctica)

Este tipo de instrucción se usa para las operaciones aritmético-lógicas entre registros. Un ejemplo de sus operaciones son add(suma), sub(resta), and, or, nor y slt(set less than).
Los 32 bits de la instrucción se dividen de la siguiente manera:

[opcode	rs	rt	rd	shamt	funct]
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

- **rs, rt:** Registros fuente.
- **rd:** Registro destino.
- **funct:** Código funcional que especifica la operación.
- Se decodifican los campos y se envían **rs** y **rt** a la ALU, el resultado de la operación especificada en **function** se escribe en la dirección de memoria **rd**.

Tipo I (inmediato)

Estas instrucciones se usan para operaciones con operandos inmediatos (constantes), cargas/almacenamientos de memoria. Ejemplos de sus operaciones son addi(suma inmediata), lw(cargar palabra), sw(almacenar palabra) y beq(saltar si es igual).

Su formato de 32 bits es el siguiente:

[opcode	rs	rt	inmediato]
6 bits	5 bits	5 bits	16 bits

- **rs:** Registro fuente.
- **rt:** Registro destino o fuente (depende de la instrucción).
- **inmediato:** Valor constante de 16 bits (o 12 en RISC-V), a menudo extendido con signo.
- El valor **inmediato** se extiende en signo (sign-extend) y se utiliza junto con **rs** en la ALU.

Tipo J (salto/jump)

En este tipo las instrucciones usan saltos incondicionales a direcciones de memoria. Ejemplo de sus operandos son j(jump) y jal(jump and link).
El formato de instrucción a 32 bits es:

[opcode	dirección/objetivo]
6 bits	26 bits

- **dirección:** Campo de 26 bits que, combinado con el PC (Program Counter), define la dirección de salto.
- La unidad de control actualiza el PC directamente utilizando la **dirección** de 26 bits.

2. Objetivos

2.1 Objetivo general

Explicar el funcionamiento del Single Datapath, tomando en cuenta todos su elementos, tales como, el banco de registros, ALU, multiplexores, unidades de control, desde la investigación, su diseño, implementación y ejecución, mediante el uso de instrucciones de tipo R.

2.1 Objetivos específicos

- Investigar de manera profunda cada concepto para su entendimiento y aplicación.
- Comprender el formato de instrucción tipo R y las diferencias con otros.
- Desglosar como parte del diseño en diferentes módulos para su creación.
- Implementar cada módulo requerido en Modelsim.
- Realizar los Test Bench necesarios para comprobar el funcionamiento de la práctica completa.

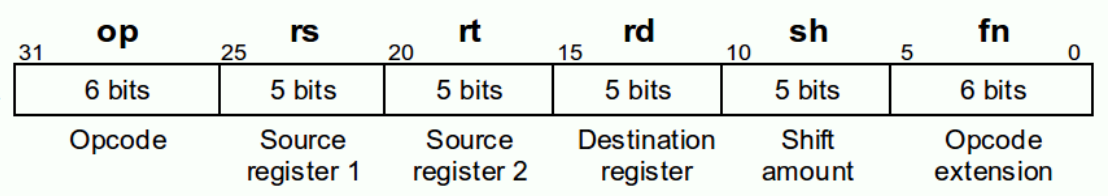
3. Marco Teórico

3.1 Instrucción Tipo R

En la arquitectura MIPS, las instrucciones tipo R (Register-type) se utilizan para realizar operaciones entre registros. Este tipo de instrucción se caracteriza por operar exclusivamente con datos almacenados en registros internos del procesador, sin acceso directo a la memoria.

El formato (máquina) de instrucción tipo R se compone de los siguientes campos:

- **opcode:** código de operación (generalmente 0 en tipo R)
- **rs:** primer registro fuente
- **rt:** segundo registro fuente
- **rd:** registro destino
- **shamt:** desplazamiento (shift amount)
- **funct:** especifica la operación exacta



Representación de la separación de bits para cada campo.

Como se observa en la imagen, de los 32 bits provenientes de la instrucción el campo function y el opcode necesitan de 6 bits para llevar a cabo la operación, y los campos restantes constan de 5 bits.

De acuerdo a esta información, la separación de campos en verilog para instrucciones del tipo R en formato MIPS, según los bits especificados para cada apartado, quedaría algo así:

```
assign opcode   =  instruccion[31:26];
assign rs       =  instruccion[25:21];
assign rt       =  instruccion[20:16];
assign rd       =  instruccion[15:11];
assign shamt    =  instruccion[10:6];
assign funct    =  instruccion[5:0];
```

Opcode y **funct** con 6 bits cada uno, **rs**, **rt**, **rd** y **shamt** con 5 bits, dando el total de 32 bits para la instrucción.

Sintaxis para Ensamblador

La sintaxis básica para instrucciones tipo R (Registro-Registro) en lenguaje ensamblador, es la siguiente:

opcode registro_destino , registro_fuente1 , registro_fuente2

Ejemplo:

add $7 , $4 , $2

Ejemplo de la sintaxis para una suma en código ensamblador, se usa el operador add. Al número en la dirección de memoria 4 se le suma el número en la dirección de memoria 2, el resultado se escribe en la dirección de memoria 7.

Elementos de la Sintaxis

- **Mnemónico (Opcode):** Nombre de la instrucción (ej. ADD, SUB, AND, OR, SLT).
- **Registro Destino (rd):** El primer operando tras el mnemónico, donde se almacenará el resultado.
- **Registro Fuente 1 (rs):** Segundo operando, primer valor de entrada a la ALU.
- **Registro Fuente 2 (rt):** Tercer operando, segundo valor de entrada a la ALU.

Operaciones implementables

Existe una gran cantidad de operaciones aritmético-lógicas a realizar con las instrucciones tipo R, tales como add(suma), sub(resta), and, or. Dentro de estas una de las operaciones más relevantes es **SLT** (Set on Less Than). Esta instrucción compara dos registros y establece el valor del registro destino dependiendo del resultado de la comparación:

- Si **rs < rt**, entonces **rd = 1**
- En caso contrario, **rd = 0**

La instrucción SLT es fundamental para la implementación de estructuras de control en bajo nivel, como condicionales (if) y ciclos (while), ya que permite realizar comparaciones entre valores.

Internamente la ALU no puede realizar la comparativa entre un valor y otro e indicar si es menor o mayor directamente, es por eso que, esta operación suele implementarse mediante una resta (**rs - rt**) y la evaluación del bit de signo del resultado.

3.2 Operador Ternario

La operación ternaria es común en lenguajes de programación de alto nivel como C y Verilog. Se trata de una forma compacta de expresar una estructura condicional.

La sintaxis de este operador en C sería la siguiente:

condición ? valor_si_verdadero : valor_si_falso;

Esta operación evalúa una condición lógica. Si la condición es verdadera, retorna el primer valor, si es falsa, retorna el segundo.

En el contexto de diseño digital (por ejemplo, en Verilog), la operación ternaria se utiliza frecuentemente para describir lógica combinacional. Por ejemplo:

assign salida = (a < b) ? 1 : 0;

Aquí se observa una equivalencia directa con la instrucción SLT, ya que se está evaluando una condición de comparación y asignando un valor binario como resultado. Es útil, ya que simplifica el código y puede representar directamente multiplexores en hardware, lo que facilita la síntesis.

3.3 Proceso de Compilación

El proceso de compilación es el conjunto de etapas mediante las cuales un programa escrito en cualquier lenguaje se transforma en código ejecutable por una máquina. Este proceso es fundamental en la informática y está relacionado con Ingeniería de Software y Arquitectura de Computadoras.

Las principales fases del proceso de compilación son:

1. **Análisis léxico:** Se encarga de convertir el código fuente en una secuencia de tokens (palabras clave, identificadores, operadores, etc.).

2. **Análisis sintáctico:** Verifica que la estructura del programa cumpla con las reglas gramaticales del lenguaje. Aquí se detectan errores de sintaxis.

3. **Análisis semántico:** Comprueba que las operaciones tengan sentido lógico, como tipos de datos compatibles y uso correcto de variables.

4. **Generación de código intermedio:** Produce una representación intermedia del programa, independiente de la máquina.

5. **Optimización:** Mejora el código intermedio para hacerlo más eficiente en tiempo o espacio.

6. **Generación de código objeto:** Traduce el código intermedio en lenguaje máquina específico de la arquitectura.

7. **Enlazado (linking):** Combina distintos módulos y bibliotecas para generar el programa ejecutable final.

En el contexto de hardware (como en Verilog), el proceso equivalente es la síntesis, donde el código se transforma en una representación de compuertas lógicas implementables en un circuito.

4. Desarrollo

A continuación, se muestra la implementación en Verilog de cada uno de los módulos necesarios para el funcionamiento del sistema, además de la simulación.

Top Module DPTR

```
module DPTR (
    input clk,
    input [31:0] instruction,
    output [31:0] result
);
    wire [5:0] op = instruction[31:26];
    wire [4:0] rs = instruction[25:21];
    wire [4:0] rt = instruction[20:16];
    wire [4:0] rd = instruction[15:11];
    wire [5:0] funct = instruction[5:0];

    wire memToReg, RegWrite, memToWrite, memToRead;
    wire [2:0] ALUOp, ALUCtrl;
    wire [31:0] readData1, readData2, aluResult;

    U_Control UC(op, memToReg, memToWrite, memToRead, ALUOp, RegWrite);
    ALUControl ALUC(funct, ALUOp, ALUCtrl);
    BancoReg BR(clk, RegWrite, rs, rt, rd, aluResult, readData1, readData2);
    ALU alu(readData1, readData2, ALUCtrl, aluResult);

    assign result = aluResult;
endmodule
```

```
Ln#
 1  module DPTR (
 2      input clk,
 3      input [31:0] instruction,
 4      output [31:0] result
 5  );
 6      wire [5:0] op = instruction[31:26];
 7      wire [4:0] rs = instruction[25:21];
 8      wire [4:0] rt = instruction[20:16];
 9      wire [4:0] rd = instruction[15:11];
10      wire [5:0] funct = instruction[5:0];
11
12      wire memToReg, RegWrite, memToWrite, memToRead;
13      wire [2:0] ALUOp, ALUCtrl;
14      wire [31:0] readData1, readData2, aluResult;
15
16      U_Control UC(op, memToReg, memToWrite, memToRead, ALUOp, RegWrite);
17      ALUControl ALUC(funct, ALUOp, ALUCtrl);
18      BancoReg BR(clk, RegWrite, rs, rt, rd, aluResult, readData1, readData2);
19      ALU alu(readData1, readData2, ALUCtrl, aluResult);
20
21      assign result = aluResult;
22  endmodule
```

Captura del código en ModelSim.

Módulo ALU

```
module ALU (
    input [31:0] A, B,
    input [2:0] ALUCtrl,
    output reg [31:0] Result
);
    always @(*) begin
        case (ALUCtrl)
            3'b010: Result = A + B;          // SUMA
            3'b110: Result = A - B;          // RESTA
            3'b000: Result = A & B;          // AND
            3'b001: Result = A | B;          // OR
            3'b111: Result = (A < B) ? 1:0;  // SLT
            default: Result = 0;
        endcase
    end
endmodule
```

```
Ln#
 1  module ALU (
 2      input [31:0] A, B,
 3      input [2:0] ALUCtrl,
 4      output reg [31:0] Result
 5  );
 6      always @(*) begin
 7          case (ALUCtrl)
 8              3'b010: Result = A + B;          // SUMA
 9              3'b110: Result = A - B;          // RESTA
10              3'b000: Result = A & B;          // AND
11              3'b001: Result = A | B;          // OR
12              3'b111: Result = (A < B) ? 1:0;  // SLT
13              default: Result = 0;
14          endcase
15      end
16  endmodule
```

Captura de código para módulo ALU en ModelSim.

Módulo ALU Control

```
module ALUControl (
    input [5:0] fnc,
    input [2:0] ALUOp,
    output reg [2:0] Sel
);
    always @(*) begin
        case (ALUOp)
            3'b010: begin
                case (fnc)
                    6'b100000: Sel = 3'b010; // Suma
                    6'b100010: Sel = 3'b110; // Resta
                    6'b100100: Sel = 3'b000; // And
                    6'b100101: Sel = 3'b001; // Or
```

```
                    6'b101010: Sel = 3'b111; // SLT
            default:   Sel = 3'b000;
        endcase
        end
      default: Sel = 3'b000;
        endcase
    end
endmodule
```

```
Ln#
 1  module ALUControl (
 2      input [5:0] fnc,
 3      input [2:0] ALUOp,
 4      output reg [2:0] Sel
 5  );
 6      always @(*) begin
 7          case (ALUOp)
 8              3'b010: begin
 9                  case (fnc)
10                      6'b100000: Sel = 3'b010; // Suma
11                      6'b100010: Sel = 3'b110; // Resta
12                      6'b100100: Sel = 3'b000; // And
13                      6'b100101: Sel = 3'b001; // Or
14                      6'b101010: Sel = 3'b111; // SLT
15                   default:   Sel = 3'b000;
16              endcase
17                  end
18           default: Sel = 3'b000;
19           endcase
20      end
21  endmodule
```

Captura del código para módulo ALU Control en ModelSim.

Modulo Unidad de Control

```
module U_Control(
        input [5:0]op,
        output reg memToReg,
        output reg memToWrite,
        output reg memToRead,
        output reg [2:0]AluOP,
        output reg RegWrite
);
always@(*)
begin
        RegWrite = 1'b0;
        AluOP = 3'b000;
        memToRead =1'b0;
        memToWrite =1'b0;
        memToReg =1'b0;

case (op)
        6'b000000:
        begin
```

```
                RegWrite = 1'b1;
                AluOP = 3'b010;
                memToRead =1'b0;
                memToWrite =1'b0;
                memToReg =1'b0;
        end
endcase
end
endmodule
```

```
Ln#
 1  module U_Control(
 2          input [5:0]op,
 3          output reg memToReg,
 4          output reg memToWrite,
 5          output reg memToRead,
 6          output reg [2:0]AluOP,
 7          output reg RegWrite
 8  );
 9  always@(*)
10  begin
11          RegWrite = 1'b0;
12          AluOP = 3'b000;
13          memToRead =1'b0;
14          memToWrite =1'b0;
15          memToReg =1'b0;
16
17  case (op)
18          6'b000000:
19          begin
20                  RegWrite = 1'b1;
21                  AluOP = 3'b010;
22                  memToRead =1'b0;
23                  memToWrite =1'b0;
24                  memToReg =1'b0;
25          end
26  endcase
27  end
28  endmodule
```

Captura del código para el módulo Unidad de Control en ModelSim.

Módulo Banco de Registros

```
module BancoReg (
    input clk,
    input RegWrite,
    input [4:0] rs, rt, rd,
    input [31:0] writeData,
    output [31:0] readData1, readData2
);
    reg [31:0] regs [0:31];

    assign readData1 = regs[rs];
    assign readData2 = regs[rt];

    always @(posedge clk) begin
```

```
    if (RegWrite && rd != 0)
        regs[rd] <= writeData;
end
endmodule
```

```
Ln#
1   module BancoReg (
2       input clk,
3       input RegWrite,
4       input [4:0] rs, rt, rd,
5       input [31:0] writeData,
6       output [31:0] readData1, readData2
7   );
8       reg [31:0] regs [0:31];
9
10      assign readData1 = regs[rs];
11      assign readData2 = regs[rt];
12
13      always @(posedge clk) begin
14      if (RegWrite && rd != 0)
15          regs[rd] <= writeData;
16  end
17
18  endmodule
```

Código en Verilog del módulo Banco de Registros en ModelSim.

Módulo Memoria

```
module Mem (
    input clk,
    input MemWrite,
    input [31:0] addr,
    input [31:0] write,
    output reg [31:0] read
);
    reg [31:0] memory [0:255];

    always @(*) begin
    read = memory[addr];
end

always @(posedge clk) begin
    if (MemWrite)
        memory[addr] <= write;
end
endmodule
```

```
Ln#
1   module Mem (
2       input clk,
3       input MemWrite,
4       input [31:0] addr,
5       input [31:0] write,
6       output reg [31:0] read
7   );
8       reg [31:0] memory [0:255];
9
10      always @(*) begin
11      read = memory[addr];
12  end
13
14  always @(posedge clk) begin
15      if (MemWrite)
16          memory[addr] <= write;
17  end
18
19  endmodule
20  ////////UNION DE MEMORIA DE DATOS Y MEMORIA DE INSTRUCCIONES
```

Captura de código para Memoria en ModelSim.

Módulo Multiplexor

```
module Mux2_1_32 (
    input [31:0] in0, in1,
    input sel,
    output [31:0] out
);
    assign out = sel ? in1 : in0;
endmodule
```

```
Ln#
1   module Mux2_1_32 (
2       input [31:0] in0, in1,
3       input sel,
4       output [31:0] out
5   );
6       assign out = sel ? in1 : in0;
7   endmodule
```

Captura de código para el MUX en ModelSim.

Módulo TestBench

```
module TB_DPTR;
    reg clk;
    reg [31:0] instruction;
    wire [31:0] result;

    DPTR tb (
        .clk(clk),
        .instruction(instruction),
        .result(result)
```

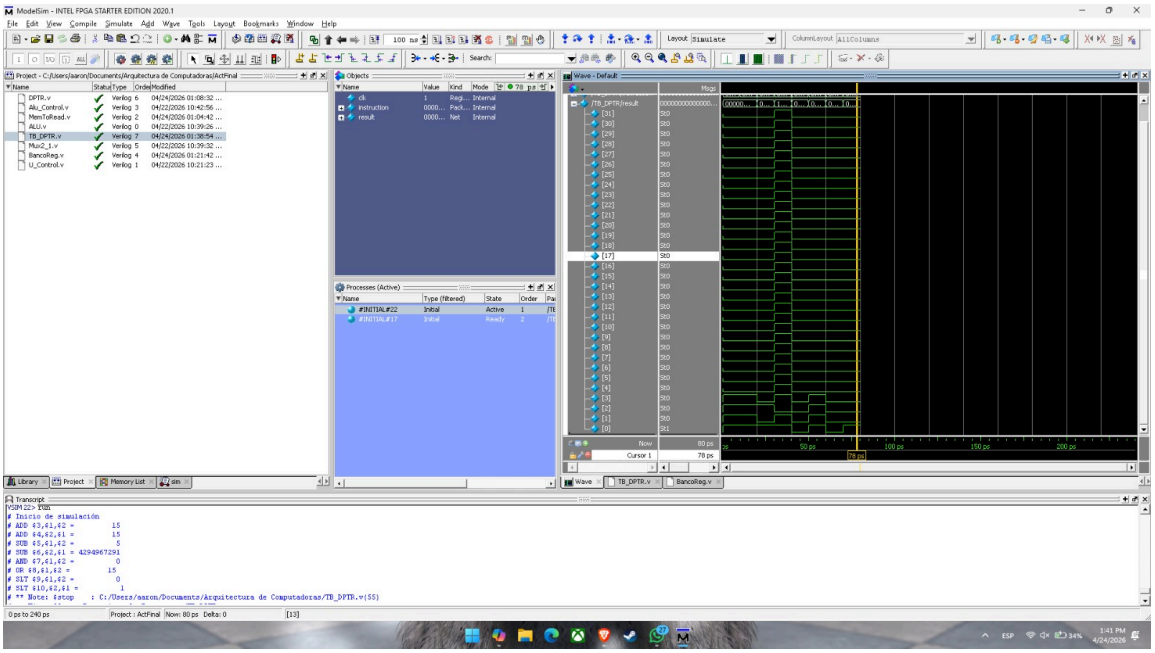
```
    );
    initial begin
        tb.BR.regs[1] = 32'd10;
        tb.BR.regs[2] = 32'd5;
        tb.BR.regs[3] = 32'd0;
    end

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $display("Inicio de simulación");

            // SUMA
            instruction = {6'b000000, 5'd1, 5'd2, 5'd3, 5'd0, 6'b100000};
            #10 $display("ADD $3,$1,$2 = %d", result);

            instruction = {6'b000000, 5'd2, 5'd1, 5'd4, 5'd0, 6'b100000};
            #10 $display("ADD $4,$2,$1 = %d", result);

            // RESTA
            instruction = {6'b000000, 5'd1, 5'd2, 5'd5, 5'd0, 6'b100010};
            #10 $display("SUB $5,$1,$2 = %d", result);

            instruction = {6'b000000, 5'd2, 5'd1, 5'd6, 5'd0, 6'b100010};
            #10 $display("SUB $6,$2,$1 = %d", result);

            // AND
            instruction = {6'b000000, 5'd1, 5'd2, 5'd7, 5'd0, 6'b100100};
            #10 $display("AND $7,$1,$2 = %d", result);

            // OR
            instruction = {6'b000000, 5'd1, 5'd2, 5'd8, 5'd0, 6'b100101};
            #10 $display("OR $8,$1,$2 = %d", result);

            // SLT
            instruction = {6'b000000, 5'd1, 5'd2, 5'd9, 5'd0, 6'b101010};
            #10 $display("SLT $9,$1,$2 = %d", result);

            instruction = {6'b000000, 5'd2, 5'd1, 5'd10, 5'd0, 6'b101010};
            #10 $display("SLT $10,$2,$1 = %d", result);


        $stop;
    end
endmodule
```

```
Ln#
1  module TB_DPTR;
2      reg clk;
3      reg [31:0] instruction;
4      wire [31:0] result;
5
6      DPTR tb (
7          .clk(clk),
8          .instruction(instruction),
9          .result(result)
10     );
11     initial begin
12         tb.BR.regs[1] = 32'd10;
13         tb.BR.regs[2] = 32'd5;
14         tb.BR.regs[3] = 32'd0;
15     end
16
17     initial begin
18         clk = 0;
19         forever #5 clk = ~clk;
20     end
21
22     initial begin
23         $display("Inicio de simulación");
24
25         // SUMA
26         instruction = {6'b000000, 5'd1, 5'd2, 5'd3, 5'd0, 6'b100000};
27         #10 $display("ADD $3,$1,$2 = %d", result);
28
29         instruction = {6'b000000, 5'd2, 5'd1, 5'd4, 5'd0, 6'b100000};
30         #10 $display("ADD $4,$2,$1 = %d", result);
31
32         // RESTA
33         instruction = {6'b000000, 5'd1, 5'd2, 5'd5, 5'd0, 6'b100010};
34         #10 $display("SUB $5,$1,$2 = %d", result);
35
36         instruction = {6'b000000, 5'd2, 5'd1, 5'd6, 5'd0, 6'b100010};
37         #10 $display("SUB $6,$2,$1 = %d", result);
```

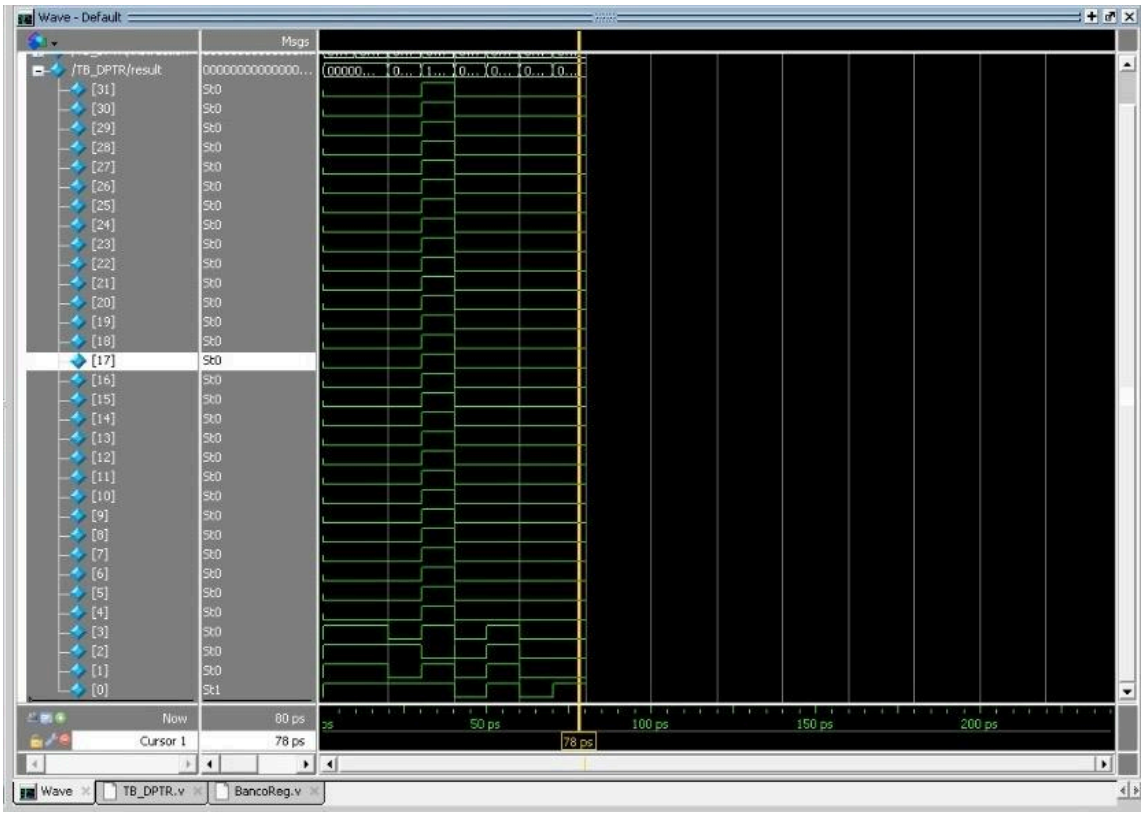
```
39         // AND
40         instruction = {6'b000000, 5'd1, 5'd2, 5'd7, 5'd0, 6'b100100};
41         #10 $display("AND $7,$1,$2 = %d", result);
42
43         // OR
44         instruction = {6'b000000, 5'd1, 5'd2, 5'd8, 5'd0, 6'b100101};
45         #10 $display("OR $8,$1,$2 = %d", result);
46
47         // SLT
48         instruction = {6'b000000, 5'd1, 5'd2, 5'd9, 5'd0, 6'b101010};
49         #10 $display("SLT $9,$1,$2 = %d", result);
50
51         instruction = {6'b000000, 5'd2, 5'd1, 5'd10, 5'd0, 6'b101010};
52         #10 $display("SLT $10,$2,$1 = %d", result);
53
54
55         $stop;
56     end
57 endmodule
```

Capturas de código para el Test Bench en ModelSim.

4.1 Capturas Simulación



Captura donde se observan todos los archivos .v para el proyecto, y la ventana Wave con las señales correspondientes a cada operación realizada por el Test Bench, 10 pruebas en total, 2 por cada operación.



Captura ampliada de la ventana wave, en la parte de abajo se muestran las señales para cada operación.

Memory Data - /TB_DPTR/tb/BR/regs - Default

0	x
1	10
2	5
3	15
4	15
5	5
6	-5
7	0
8	15
9	0
10	1
11	x
12	x
13	x
14	x
15	x
16	x
17	x
18	x
19	x
20	x
21	x
22	x
23	x
24	x
25	x
26	x
27	x
28	x
29	x
30	x
31	x

Captura de ventana Memory data con los resultados para cada operación en decimal.

Desglose de resultados para cada dirección

Dirección	Operación	Valores	Resultado
1	Reservada -> imprime valor del operando 1	10	10
2	Reservada -> imprime valor del operando 2	5	5
3	ADD $3,$1,$2 = %d	10 + 5	15
4	ADD $4,$2,$1 = %d	5 + 10	15
5	SUB $5,$1,$2 = %d	10 - 5	5
6	SUB $6,$2,$1 = %d	5 - 10	-5
7	AND $7,$1,$2 = %d	10 & 5	0
8	OR $8,$1,$2 = %d	10 | 5	15
9	SLT $9,$1,$2 = %d	(10<5) ? 1:0	0
10	SLT $10,$2,$1 = %d	(5<10) ? 1:0	1

5. Conclusión

A partir del desarrollo de esta práctica, se logró una comprensión integral del funcionamiento de un procesador basado en un Single Datapath, analizando no solo su estructura interna, sino también la interacción real entre cada uno de sus componentes. El trabajo permitió pasar de un entendimiento teórico a una aplicación práctica, demostrando cómo elementos como la ALU, la unidad de control y el banco de registros se coordinan para ejecutar instrucciones de manera eficiente dentro de un solo ciclo de reloj.

Durante el proceso, aplicamos capacidad de interpretar y transformar especificaciones de la arquitectura en módulos funcionales, lo cual implicó tomar decisiones de diseño, resolver problemas de integración y validar el comportamiento mediante simulaciones. Esto también facilitó comprender la importancia de la correcta decodificación de instrucciones y del control de señales, aspectos clave para garantizar que el flujo de datos opere de forma coherente.

Asimismo, la implementación en Verilog y el uso de herramientas como ModelSim permitieron desarrollar habilidades en descripción de hardware y verificación, demostrando la relevancia de los testbench como medio para comprobar el correcto funcionamiento del sistema ante distintos escenarios. Más allá de los resultados obtenidos, el equipo adquirió una visión más clara de cómo se materializan los conceptos de arquitectura de computadoras en soluciones reales.

En esta práctica se cumplió con los objetivos planteados, y se aportó una base sólida para abordar diseños más complejos, por medio del trabajo colaborativo y el pensamiento lógico necesario en el área de sistemas digitales.

6. Bibliografía

- Wikipedia. (2025). MIPS architecture. Wikipedia. Recuperado de https://en.wikipedia.org/wiki/MIPS_architecture

- GeeksforGeeks. (2024). Instruction format in MIPS. Recuperado de https://www.geeksforgeeks.org/instruction-format-in-mips/

- GeeksforGeeks. (2024). Conditional or ternary operator in programming. Recuperado de https://www.geeksforgeeks.org/conditional-or-ternary-operator-in-programming/

- Patterson, D. A., & Hennessy, J. L. (2017). Computer organization and design MIPS edition: The hardware/software interface (5th ed.). Morgan Kaufmann.

- Mano, M. M., & Ciletti, M. D. (2017). Digital design (6th ed.). Pearson.

- TutorialsPoint. (2024). Compiler design phases. Recuperado de https://www.tutorialspoint.com/compiler_design/compiler_design_phases.htm

- Nandland. (2023). Verilog conditional operator. Recuperado de https://nandland.com/conditional-operator/